

ConvStem-MAE：通过卷积 Patch 嵌入与掩码自编码器预训练改进 Vision Transformer

1. 引言

1.1 从 NLP 到 CV：Transformer 的跨界之旅

Transformer 架构最初是为自然语言处理（NLP）设计的。其核心机制——自注意力（Self-Attention）——使模型能够捕捉序列中任意两个位置之间的依赖关系，彻底改变了 NLP 领域。基于 Transformer 的两大范式在 NLP 中取得了巨大成功：

- 掩码语言建模（Masked Language Modeling, MLM）**：以 BERT 为代表，随机遮盖句子中的部分词语，让模型根据上下文"填空"。这种"完形填空"式的训练迫使模型理解语言的深层结构。
- 自回归生成（Autoregressive Generation）**：以 GPT 系列为代表，根据已生成的内容预测下一个词，最终演化出 ChatGPT 等强大的生成式对话系统。

Vision Transformer（ViT）[Dosovitskiy et al., 2020] 证明了 Transformer 同样可以处理图像——将图像分割为 patch，每个 patch 就像一个"视觉词语"（visual word），送入 Transformer 编码器进行分类。这引出了一个自然而深刻的类比：

NLP 领域		计算机视觉领域
文本 → 分词 → 词嵌入		图像 → 分 patch → patch 嵌入
句子 = 词的序列		图像 = patch 的序列
掩码语言建模（BERT）	→	掩码图像建模（MAE）
"完形填空"：预测被遮盖的词语	→	"视觉完形填空"：重建被遮盖的像素
根据上下文生成缺失内容	→	根据可见区域"想象"被遮挡区域

既然 Transformer 能在 NLP 中通过"遮盖-预测"的方式学习语言的深层语义，那么同样的思路是否适用于图像？掩码自编码器（Masked Autoencoder, MAE）[He et al., 2022] 正是对这一问题的回答——随机遮盖图像中 60% 的 patch，训练模型根据可见部分"想象"并重建被遮盖的内容。这种**视觉"完形填空"**任务迫使模型理解图像的整体结构和语义，而非仅仅记忆局部像素模式。

这一类比之所以成立，正是因为 ViT 将图像转化为了与文本相同的"序列"形式：patch 就是视觉世界的"词语"，而重建被遮盖的 patch 就等同于补全被遮盖的词语。

1.2 ViT 的局限与改进动机

尽管 ViT 在大规模数据上表现优异，它存在一个根本性局限：**缺乏局部归纳偏置**。与 CNN 通过卷积核天然捕捉局部空间模式不同，ViT 的线性 patch 投影将每个 patch 视为独立的 token，忽略了 patch 内部和 patch 之间的丰富局部结构（边缘、纹理、空间邻接关系）。

这一局限在以下场景中尤为突出：

- 训练数据有限（如 CIFAR-10 仅 5 万张图片）
- 图像分辨率小（如 32×32），每个 patch 仅包含极少量像素

- 模型必须从零开始学习局部特征提取

这一局限不仅影响分类性能，同样影响 MAE 预训练中的重建质量——如果 patch 嵌入无法有效提取局部特征，编码器从可见 patch 中获取的信息就不够丰富，"视觉完形填空"的效果也会受限。

1.3 研究动机与核心假设

基于以上分析，本项目从**架构**和**预训练范式**两个维度对 ViT 进行改进：

架构层面：将轻量级卷积干网络（Convolutional Stem）作为 patch 嵌入模块，为 ViT 注入局部归纳偏置。

预训练层面：借鉴 NLP 中掩码语言建模的思想，通过 MAE 自监督预训练让模型学习"视觉完形填空"能力，再迁移到下游分类任务。

核心假设：将 ConvStem patch 嵌入与 MAE 自监督预训练相结合，能够产生协同效应——卷积归纳偏置不仅改善监督分类效果，还能帮助 MAE 编码器在"视觉完形填空"任务中学习到更优质的视觉表征，两者结合时在下游任务上取得最佳表现。

1.4 贡献

- ConvStem-ViT**：用轻量级多层卷积干网络替代 ViT 中的线性 patch 投影，逐步下采样输入的同时提取局部特征。
- ConvStem MAE 预训练**：将 MAE 预训练中的编码器 patch 嵌入扩展为 ConvStem 版本，验证卷积归纳偏置对"视觉完形填空"学习效果的提升。
- 跨数据集迁移**：在 Tiny ImageNet（200 类，64×64，110K 图像）上进行 MAE 自监督预训练，迁移到 CIFAR-10（10 类，32×32，60K 图像），证明通过"视觉完形填空"学到的表征可跨数据集和跨分辨率泛化。
- 完整消融实验**：进行 2×2 消融研究（ConvStem vs 线性 × 从零训练 vs MAE 预训练），揭示每项技术的单独贡献和组合效应。

2. 方法

2.1 模型架构概览

我们对比三种模型架构：

模型	Patch 嵌入方式	参数量
ResNet-18	全卷积网络	~11M
Vanilla ViT	线性投影（Conv2d, kernel=patch_size, stride=patch_size）	21.34M
ConvStem-ViT	多层 CNN 干网络	21.99M

所有 ViT 变体共享相同的 Transformer 编码器超参数，以确保公平对比。

2.2 Vanilla ViT（线性 Patch 嵌入）

原始 ViT 将 H×W 的图像分割为 P×P 大小的非重叠 patch：

图像 $(C, H, W) \rightarrow \text{Conv2d}(C, D, \text{kernel}=P, \text{stride}=P) \rightarrow \text{reshape} \rightarrow (N, H/P \times W/P, D)$

对于 CIFAR-10 (32×32)，patch_size=4，生成 8×8=64 个维度为 D 的 token。这种方法简洁高效但缺乏局部感受野——每个 patch 被独立投影，不考虑相邻 patch 的信息。

2.3 ConvStem-ViT (卷积 Patch 嵌入)

我们用多层卷积干网络替代单一线性投影：

图像 $(3, 32, 32)$
→ Conv2d(3, 192, kernel=3, stride=2, padding=1) → BatchNorm → GELU → (192, 16, 16)
→ Conv2d(192, 384, kernel=3, stride=2, padding=1) → BatchNorm → GELU → (384, 8, 8)
→ reshape → $(N, 64, 384)$

ConvStem 的优势：

- **局部感受野**：每个输出 token 聚合了局部邻域的信息，而非仅来自单一固定 patch
- **渐进下采样**：两次 stride-2 卷积替代一次 stride-4 投影，保留了更多空间细节
- **批归一化**：稳定早期训练过程
- **仅增加 0.65M 参数**：相比 21.34M 的 Vanilla ViT 开销极小

2.4 掩码自编码器 (MAE)：从"完形填空"到"视觉完形填空"

2.4.1 动机：NLP 掩码语言建模的视觉类比

在 NLP 中，BERT 通过掩码语言建模取得了巨大成功：随机遮盖句子中 15% 的词语，训练模型根据上下文预测被遮盖的词。这种训练方式之所以有效，是因为它迫使模型理解语言的深层结构和语义关系，而非简单记忆。

ViT 将图像分割为 patch 序列后，图像和文本在形式上变得统一——都是 token 序列。因此，一个自然的想法是：**能否对图像做同样的"完形填空"？** 掩码自编码器 (MAE) 正是这一思想的实现：

NLP "完形填空" (BERT)：

"[MASK] 是一种常见的[MASK]动物"

上下文 = "是"、"一种"、"常见的"、"动物"

预测："猫"、"常见"

视觉 "完形填空" (MAE)：

✓	✗	✓	✗
✗	✓	✗	✓
✓	✗	✓	✗

✓ = 可见 patch ✗ = 被遮盖 patch
根据 ✓ 重建 ✗ 的像素内容

与 BERT 遮盖 15% 不同，MAE 遮盖高达 60% 的 patch。这是因为在图像中，相邻 patch 之间存在高度冗余——一个 patch 的内容很大程度上可以从周围 patch 推断。更高的遮盖比例迫使模型真正理解图像的全局语义结构。

2.4.2 MAE 具体流程

1. **掩码化**：随机遮盖 60% 的图像 patch
2. **编码**：仅将可见（未被遮盖的）patch 通过 ViT 编码器（丢弃被遮盖的 patch，大幅节省计算量）
3. **解码**：将编码后的可见 token 与可学习的掩码 token 拼接，恢复原始顺序，通过轻量级解码器
4. **重建**：预测被遮盖 patch 的像素值
5. **损失**：仅在遮盖 patch 上计算预测值与归一化像素值之间的 MSE

2.4.3 ConvStem MAE（我们的扩展）

我们将 MAE **编码器** 中的线性 patch 嵌入替换为 ConvStem：

MAE 编码器（线性）：

图像 → 线性 PatchEmbed → [可见 token] → Transformer → 隐表示

MAE 编码器（ConvStem）：

图像 → ConvStem PatchEmbed → [可见 token] → Transformer → 隐表示

直觉上，在做“视觉完形填空”时，ConvStem 能让模型更仔细地“阅读”每一个可见的 patch——通过卷积操作提取更丰富的局部细节（边缘、纹理），而不是像线性投影那样粗略地将整个 patch 压缩为一个向量。更精细的“阅读”带来更准确的“推理”，最终产生更准确的“填空”结果。

解码器保持不变（使用简单线性投影重建 patch），因为重建质量主要取决于编码器提取的表征质量。

2.5 跨数据集迁移策略

我们的迁移学习流程：

阶段一：在 Tiny ImageNet 上进行 MAE 预训练

- 数据集：Tiny ImageNet（200 类，64×64，100K 训练 + 10K 验证）
- 任务：自监督“视觉完形填空”（不使用标签）
- 训练时长：200 个 epoch，每个模型约 1.5-2.5 小时
- 产出：预训练编码器权重

阶段二：在 CIFAR-10 上微调

- 数据集：CIFAR-10（10 类，32×32，50K 训练 + 10K 测试）
- 权重迁移：
 - 编码器块（blocks → encoder）：直接复制
 - 位置编码（pos_embed）：兼容（相同网格 8×8）
 - cls_token：直接复制
 - patch_embed：重新初始化（分辨率不同：64→32）
 - 分类头：随机初始化（200→10 类）
- 策略：冻结编码器 10 个 epoch → 解冻全部参数训练 40 个 epoch
- 训练时长：50 个 epoch，每个模型约 17 分钟

为什么可行： Tiny ImageNet 使用 `patch_size=8` 处理 64×64 图像 $\rightarrow 8\times 8=64$ 个 patch。CIFAR-10 使用 `patch_size=4` 处理 32×32 图像 $\rightarrow 8\times 8=64$ 个 patch。相同的网格大小意味着位置编码和编码器块权重可以直接迁移。

3. 实验设置

3.1 数据集

数据集	类别数	分辨率	训练集	测试集	磁盘大小
CIFAR-10	10	32×32	50,000	10,000	~170 MB
Tiny ImageNet	200	64×64	100,000	10,000	~240 MB

3.2 模型超参数

所有 ViT 模型（Vanilla 和 ConvStem）共享相同的 Transformer 编码器配置：

参数	值
嵌入维度	384
编码器深度	12 层
注意力头数	6
MLP 比率	4.0
Patch 大小（CIFAR-10）	4
Patch 大小（Tiny ImageNet）	8
网格大小	$8\times 8 = 64$ 个 token

MAE 解码器配置：

参数	值
解码器嵌入维度	192
解码器深度	4 层
解码器注意力头数	3
掩码比例	0.6

3.3 训练超参数

MAE 预训练（阶段一）：

参数	值
优化器	AdamW ($\beta_1=0.9, \beta_2=0.95$)

参数	值
学习率	5×10^{-4}
权重衰减	0.05
批大小	512
训练轮数	200
学习率调度	线性预热（20 个 epoch）+ 余弦衰减

分类训练 / 微调（阶段二）：

参数	值
优化器	AdamW
学习率	3×10^{-4}
权重衰减	0.05
批大小	128
训练轮数	50
学习率调度	余弦退火
冻结轮数（仅微调）	10

3.4 数据增强

增强方式	CIFAR-10	Tiny ImageNet
随机裁剪（padding=4）	✔ 32×32	✔ 64×64
随机水平翻转	✔	✔
标准化（均值/标准差）	CIFAR-10 统计量	ImageNet 统计量

3.5 硬件环境

- GPU：2 块 NVIDIA GeForce RTX 4090（各 24GB 显存）
- 框架：PyTorch 2.5.1 + CUDA 12.1

4. 实验结果

4.1 主实验结果 — CIFAR-10 分类

编号	方法	Patch 嵌入	预训练	测试准确率	参数量	最佳 Epoch	训练时间
①	Vanilla ViT	线性投影	无（从零训练）	74.40%	21.34M	39	19.3 分钟

编号	方法	Patch 嵌入	预训练	测试准确率	参数量	最佳 Epoch	训练时间
②	ConvStem-ViT	卷积干网络	无（从零训练）	83.09%	21.99M	46	19.7 分钟
③	Vanilla ViT	线性投影	MAE (Tiny ImageNet)	88.70%	21.34M	50	16.6 分钟
④	ConvStem-ViT	卷积干网络	MAE (Tiny ImageNet)	90.17%	21.99M	49	17.0 分钟

4.2 消融实验 — 2×2 分析

Patch 嵌入	无预训练	MAE 预训练	预训练增益 Δ
线性投影	74.40%	88.70%	+14.30%
ConvStem	83.09%	90.17%	+7.08%
ConvStem 增益 Δ	+8.69%	+1.47%	—

关键发现：

- 1. **单独 ConvStem 的效果**（行效应）：从零训练时提升准确率 **+8.69%**（74.40 → 83.09）。卷积归纳偏置为小规模数据集带来了显著优势。
- 2. **单独 MAE 预训练的效果**（列效应，线性）：提升准确率 **+14.30%**（74.40 → 88.70）。在更大数据集上通过"视觉完形填空"学到的表征是单项效果最显著的技术。
- 3. **组合效果**（实验 ④）：ConvStem + MAE 达到 **90.17%**，为最佳结果。相比仅使用 MAE（88.70 → 90.17）的提升证明 ConvStem 在有预训练的情况下仍然能提供额外收益。
- 4. **边际递减效应**：有预训练时 ConvStem 的增益（+1.47%）小于无预训练时（+8.69%）。这是合理的——MAE 预训练已经学到了强大的特征，减少了架构归纳偏置的边际收益。

4.3 MAE 预训练质量："视觉完形填空"效果对比

MAE 编码器	Patch 嵌入	最终训练损失	最佳验证损失	预训练时长
线性	线性投影	0.4618	0.5157	~2 小时 23 分
ConvStem	卷积干网络	0.1958	0.2135	~1 小时 37 分

关键发现：ConvStem MAE 的验证重建损失为 **0.2135**，比线性 MAE（0.5157）**低 58.6%**。这意味着 ConvStem 编码器在"视觉完形填空"任务上的表现远优于线性编码器——它能更准确地从可见 patch 推断被遮盖区域的内容。这种表征质量的优势直接转化为下游任务的更高准确率（90.17% vs 88.70%）。

值得注意的是，ConvStem MAE 的预训练速度也更快（1 小时 37 分 vs 2 小时 23 分），这可能是因为卷积层的局部性使得优化景观更加平滑。

4.4 收敛性分析

从零训练（实验①②）：

- Vanilla ViT 从 ~36% 准确率起步，缓慢上升，约在第 35 个 epoch 达到平台期
- ConvStem-ViT 从 ~50% 准确率起步（更高的初始性能），收敛更快
- 两个模型都出现了明显的过拟合（ConvStem 的训练准确率 ~97% vs 测试准确率 ~83%）

MAE 微调（实验③④）：

- 解冻后第一个 epoch（第 11 个 epoch）两个模型就跳至 ~55-65% 准确率
- 快速收敛：第 20 个 epoch 即达到 85%+
- 相比从零训练，过拟合程度更轻
- ConvStem 在整个训练过程中保持约 1-2% 的稳定优势

4.5 可视化结果

以下可视化结果已生成：

编号	文件	描述
图 1	results/accuracy_curves.png	4 组实验的测试准确率随 epoch 变化对比
图 2	results/loss_curves.png	所有模型的训练和测试损失曲线
图 3	results/convergence_speed.png	收敛速度对比（前 20 个 epoch 放大）
图 4	results/params_vs_accuracy.png	参数量与准确率双轴柱状图
图 5	results/mae_reconstruction_linear.png	线性 MAE 在 Tiny ImageNet 上的"视觉完形填空"效果
图 6	results/mae_reconstruction_convstem.png	ConvStem MAE 在 Tiny ImageNet 上的"视觉完形填空"效果
图 7	results/mae_error_heatmap.png	逐像素重建误差热力图（两种编码器对比）

5. 分析与讨论

5.1 为什么 ConvStem 有效？

线性 patch 嵌入将每个 4×4 patch 视为一个独立的 48 维向量。对于 32×32 的 CIFAR-10 图像，每个 patch 仅包含 16 个像素——极少量的局部上下文。ConvStem 通过以下方式解决此问题：

1. **重叠感受野**：stride-2 的 3×3 卷积创建重叠窗口，使相邻 token 共享信息
2. **层级特征提取**：两层卷积可以学习边缘检测器和纹理模式，再进行 token 化
3. **批归一化**：稳定进入 Transformer 的特征分布

5.2 为什么 ConvStem 能帮助"视觉完形填空"?

在 60% 掩码比例的 MAE 预训练中, 仅 40% 的 patch 可见。ConvStem 从每个可见 patch 中提取更丰富特征的能力, 为编码器提供了更多信息, 从而:

- 产生更好的隐表示
- 实现更低的重建损失 (0.2135 vs 0.5157)
- 形成更可迁移的特征, 提升下游任务表现

正如 NLP 中用更好的分词器 (如 BPE) 能帮助语言模型更准确地"填空"一样, ConvStem 作为一种更精细的"视觉分词器", 帮助视觉模型更准确地完成"视觉完形填空"。

5.3 跨数据集迁移

从 Tiny ImageNet (64×64, 200 类) 成功迁移到 CIFAR-10 (32×32, 10 类) 表明:

- MAE 通过"视觉完形填空"学到的是**通用视觉表征**, 不绑定于特定类别
- 匹配的网格大小 (8×8 patch) 使得编码器块和位置编码权重可以直接迁移
- 仅需重新初始化 patch 嵌入层 (输入相关) 和分类头 (任务相关)

5.4 局限性与未来工作

1. **规模**: 两个数据集都相对较小。在 ImageNet-1K 上测试能更好地验证方法的有效性。
2. **ConvStem 设计**: 当前使用 2 层卷积。探索更深的干网络或深度可分离卷积可能进一步提升效率。
3. **掩码策略**: 我们使用随机掩码 (60%)。块状掩码或注意力引导的掩码可能产生更好的表征。
4. **从"填空"到"生成"**: 当前 MAE 只做重建 (填空), 未来可以探索类似 GPT 的自回归图像生成——按顺序逐 patch 生成图像, 这将更接近 NLP 中从 BERT 到 GPT 的演化路径。

6. 结论

本项目从 **NLP 中掩码语言建模** 的思想出发, 探索了将其迁移到视觉领域的可行性——即通过 MAE 实现**"视觉完形填空"**。在此基础上, 我们进一步提出用 ConvStem 改进 patch 嵌入, 以增强模型在"填空"任务中的特征提取能力。实验结果证明:

- 单独 ConvStem: 相比 Vanilla ViT 提升 **+8.69%** (74.40 → 83.09)
- 单独 MAE"视觉完形填空"预训练: 相比从零训练提升 **+14.30%** (74.40 → 88.70)
- **ConvStem + MAE 组合**: 相比基线提升 **+15.77%** (74.40 → **90.17%**)

核心洞察在于: ConvStem 不仅通过局部归纳偏置改善了监督分类效果, 还显著提升了"视觉完形填空"的重建质量——ConvStem MAE 的重建损失比线性 MAE 低 58.6%, 直接转化为更优的下游性能。

这些发现表明, **NLP 中的成功范式 (掩码建模) 可以有效地迁移到视觉领域**, 而架构改进 (ConvStem) 与预训练范式 (MAE) 是将 Vision Transformer 适配到资源受限场景下的互补策略。

附录

A. 可复现性

所有实验可通过以下命令复现:

```
# 阶段一：在 Tiny ImageNet 上进行 MAE 预训练（总计约 4 小时）
python pretrain_mae.py --config configs/mae_pretrain_tiny_imagenet.yaml
python pretrain_mae.py --config
configs/mae_pretrain_convstem_tiny_imagenet.yaml

# 阶段二：从零训练分类 baseline（总计约 40 分钟）
python train.py --config configs/vanilla_vit.yaml
python train.py --config configs/convstem_vit.yaml

# 阶段三：从 MAE 预训练权重微调（总计约 35 分钟）
python finetune.py --config configs/finetune_vanilla_vit_cifar10.yaml
python finetune.py --config configs/finetune_convstem_vit_cifar10.yaml

# 阶段四：生成全部可视化
python scripts/plot_all_results.py
```

或使用一键脚本：

```
bash scripts/run_transfer_learning.sh
```

B. 完整文件结构

```
deeplearning-vit/
├── configs/
│   ├── vanilla_vit.yaml           # ① Vanilla ViT 从零训练
│   ├── convstem_vit.yaml         # ② ConvStem-ViT 从零训练
│   ├── mae_pretrain_tiny_imagenet.yaml # ③a 线性 MAE 预训练
│   ├── mae_pretrain_convstem_tiny_imagenet.yaml # ④a ConvStem MAE 预训练
│   ├── finetune_vanilla_vit_cifar10.yaml # ③b 线性 MAE → CIFAR-10
│   └── finetune_convstem_vit_cifar10.yaml # ④b ConvStem MAE →
├── CIFAR-10
│   ├── src/
│   │   ├── data.py               # 数据加载（CIFAR-10 + Tiny ImageNet）
│   │   ├── models.py             # ViT、ConvStem-ViT、ResNet-18
│   │   ├── models_mae.py         # 支持线性/ConvStem 编码器的 MAE
│   │   ├── engine.py             # 分类训练循环
│   │   ├── engine_mae.py         # MAE 预训练循环
│   │   └── utils.py              # 工具函数
│   ├── train.py                  # 分类训练脚本
│   ├── finetune.py               # MAE → 分类微调脚本
│   ├── pretrain_mae.py           # MAE 预训练脚本
│   ├── evaluate.py               # 模型评估
│   ├── visualize_mae.py          # MAE 重建可视化
│   └── scripts/
│       ├── run_transfer_learning.sh # 一键完整流程
│       └── plot_all_results.py      # 生成全部图表
├── results/                      # 生成的可视化结果
└── outputs/                      # 训练 checkpoint 和指标
```

C. MAE 预训练收敛数据

Epoch	线性 MAE 训练损失	线性 MAE 验证损失	ConvStem MAE 训练损失	ConvStem MAE 验证损失
1	1.0499	0.8581	1.0201	0.8545
10	0.5400	0.5900	0.4621	0.5233
50	0.4365	0.5199	0.2466	0.2851
100	0.4470	0.5224	0.2128	0.2379
150	0.4539	0.5188	0.2001	0.2209
200	0.4618	0.5163	0.1958	0.2149